EV Fault Diagnosis RAG

Project architecture, file guide, setup, and current status

Prepared from the repository at
/Users/divyesamdani/Documents/work/ev-fault-diagnosis-rag-full-integration

Branch: feature/retrieval-first-website-integration | Latest commit: 200fcc1

Project overview

This project is an electric-vehicle fault-diagnosis website. A user describes a vehicle problem, such as a battery losing power during acceleration. The system searches technical documents, selects relevant evidence, cleans the retrieved text, and produces a structured answer using Gemini or a deterministic fallback.

The answer can include:

- Likely root cause
- Symptoms
- Possible causes
- Suggested checks
- Technical specifications
- Safety notes
- Source documents and page references
- Retrieval relevance and grounding information

The project is currently on the `feature/retrieval-first-website-integration` branch. The latest pushed commit is `200fcc1`.

Explain it simply

Think of the application as four connected workers:

1. **Website:** collects the user's question and displays the answer.
2. **Backend gateway:** directs requests to the correct service.
3. **Agent:** finds evidence, prepares it, and creates the answer.
4. **Document search:** stores technical documents as small searchable passages.

Gemini is not the document database. Gemini reads the evidence selected by the retrieval system and turns it into a clear response. This is intended to reduce unsupported answers.

Repository structure

```
ev-fault-diagnosis-rag-full-integration/
■■■ frontend/                Next.js website
■■■ backend/                 FastAPI gateway and retrieval API
■■■ agent-layer/             Agent, pipeline, DTC tools, Gemini integration
■■■ scraper/                 Document processing and vector search
■■■ data/                    Project-level data
■■■ docs/                    Documentation
■■■ literature/              Research material
■■■ dev-notes/               Local notes and evaluation reports
```

Frontend

The frontend is a Next.js application running on port `3000`.

Main files

- `frontend/src/app/page.tsx` - Dashboard and navigation.
- `frontend/src/app/diagnose/page.tsx` - Main diagnosis form and answer display.
- `frontend/src/app/dtc-lookup/page.tsx` - DTC/error-code lookup screen.
- `frontend/src/app/knowledge/page.tsx` - Searchable Knowledge Base view.
- `frontend/src/app/system/page.tsx` - Service and configuration status.
- `frontend/src/components/Shell.tsx` - Shared layout, navigation, errors, and empty states.
- `frontend/src/components/EvidenceDrawer.tsx` - Detailed evidence display.
- `frontend/src/lib/api.ts` - Central frontend API client.
- `frontend/src/lib/types.ts` - TypeScript response and request types.
- `frontend/next.config.mjs` - Rewrites `/api/v1/*` requests to the backend at port `8000`.

The browser normally calls the same frontend origin. Next.js forwards API calls to the backend gateway, so protected retrieval credentials are not placed in browser code.

Backend gateway

The backend is `backend/app.py`, a FastAPI application running on port `8000`.

It has two responsibilities:

1. Provide the authenticated vector-retrieval endpoint.
2. Forward website and agent requests to the agent API on port `8080`.

Important routes

- `GET /health` - Backend and retrieval readiness.
- `POST /api/v1/retrieval/search` - Authenticated vector search.
- `GET /api/v1/retrieval/dtc/{code}` - Authenticated DTC catalog lookup.
- Forwarded paths such as `/api/v1/query`, `/api/v1/diagnose`, `/api/v1/knowledge/search`, and `/api/v1/history`.

The DTC route supports both EV-specific entries and generic OBD-II entries. It normalizes lowercase and space-padded input, so `p0300`, `P0300`, and `P0300 ` are equivalent.

Agent and pipeline

The agent service is under `agent-layer/` and normally runs on port `8080`.

Important files

- `agent-layer/src/api/routes.py` - Agent HTTP endpoints.
- `agent-layer/src/pipeline/service.py` - Coordinates live retrieval, cleaning, summarization, and history.
- `agent-layer/src/pipeline/cleaner.py` - Removes HTML/noise and normalizes evidence.
- `agent-layer/src/pipeline/summarizer.py` - Calls Gemini or fallback generation and maps citations.
- `agent-layer/src/pipeline/history.py` - Stores local diagnosis history in SQLite.
- `agent-layer/src/mcp_server/backend_adapter.py` - Calls the backend retrieval API and handles DTC/component lookup.
- `agent-layer/src/mcp_server/tools.py` - Exposes retrieval, DTC, and component tools.
- `agent-layer/src/orchestrator/agent.py` - Direct diagnosis/orchestration flow.
- `agent-layer/src/orchestrator/mcp_client.py` - Wraps tools for LangChain.
- `agent-layer/src/common/models.py` - Pydantic contracts such as `DTCRecord`, `EvidenceSource`, and `GroundedAnswer`.
- `agent-layer/config/settings.py` - Loads provider, retrieval, MCP, and database settings.

Agent endpoints

- `GET /api/v1/health`
- `POST /api/v1/query`
- `POST /api/v1/diagnose`
- `POST /api/v1/knowledge/search`
- `GET /api/v1/dtc/{code}`
- `GET /api/v1/history`

The website's main diagnosis form uses `/api/v1/query`. The direct orchestrator path remains available through `/api/v1/diagnose`.

Retrieval and scraper layer

The retrieval implementation is under `scraper/`.

- `scraper/scraper.py` - Collects source material.
- `scraper/process-html.py` - Cleans downloaded HTML.
- `scraper/chunker.py` - Splits documents into smaller passages.
- `scraper/vector_search/retrieval.py` - Loads chunks and performs low-level search.
- `scraper/vector_search/embedding.py` - Embedding backend integration.
- `scraper/vector_search/retriever.py` - High-level ranked retrieval wrapper.

The current embedding model is:

```
sentence-transformers/all-MiniLM-L6-v2
```

The local corpus is expected under:

```
scraper/data/chunked/
```

The local embedding cache is expected at:

```
scraper/data/embeddings/chunk_vectors.json
```

These files are intentionally local and are not pushed to GitHub.

DTC lookup

The project has two classes of codes:

EV-specific codes

- `P0A80` - Severe battery cell voltage deviation.
- `E-08` - Motor Hall sensor circuit fault.
- `E-07` - Throttle signal out of range.
- `U0100` - Lost communication with the motor control module.

Generic OBD-II codes

The catalog is stored in:

```
agent-layer/data/generic_dtc_codes.json
```

It includes common codes such as:

```
P0100, P0113, P0128, P0171, P0201, P0300,
P0420, P0442, P0500, P0562, P0606, P0700,
P0A0F, P0A1A, P0A93, U0121, U0140
```

Each entry includes a description, symptoms, possible causes, suggested checks, and a public reference URL. Generic definitions are reference information; the exact meaning of a manufacturer-specific code must be verified against the vehicle's official documentation.

Complete diagnosis flow

```
Browser
  to Next.js frontend :3000
  to Backend gateway :8000
  to Agent API :8080
  to LivePipelineService
  to BackendRetrievalAdapter
  to Backend retrieval search
  to MiniLM embeddings and cosine similarity
  to Ranked document chunks
  to RetrievalCleaner
  to Gemini or deterministic fallback
  to WebsiteCardPayload
  to SQLite history
  to Browser
```

The backend retrieval request is authenticated with a local bearer token. The frontend does not receive that token.

DTC flow

```
DTC page
```

```
  to GET /api/v1/dtc/{code}
  to Backend gateway
  to Agent API
  to Backend DTC route
  to EV-specific or generic JSON catalog
  to DTCRecord
  to Frontend result card
```

Knowledge Base flow

```
Knowledge Base page
  to POST /api/v1/knowledge/search
  to Agent API
  to Backend retrieval search
  to Top five chunks
  to Full source passages and metadata in the UI
```

Local services

Service	Folder/file	Port	Purpose
Frontend	`frontend/`	3000	Website
Backend	`backend/app.py`	8000	Gateway and retrieval API
Agent	`agent-layer/`	8080	Pipeline and synthesis
MCP server	`agent-layer/src/mcp_server/`	8001 when used	Tool protocol
History	`agent-layer/data/`	N/A	Local SQLite history

How to run

Use the repository folder:

```
/Users/divyesamdani/Documents/work/ev-fault-diagnosis-rag-full-integration
```

Terminal 1: backend

```
cd /Users/divyesamdani/Documents/work/ev-fault-diagnosis-rag-full-integration
agent-layer/.venv/bin/python -m uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

Terminal 2: agent

```
cd /Users/divyesamdani/Documents/work/ev-fault-diagnosis-rag-full-integration/agent-layer
HYBRID_RETRIEVAL_USE_MOCK=false .venv/bin/python scripts/run_orchestrator_api.py
```

Terminal 3: frontend

```
cd /Users/divyesamdani/Documents/work/ev-fault-diagnosis-rag-full-integration/frontend
npm run dev
```

Open:

```
http://127.0.0.1:3000
```

Verification commands

Check backend health:

```
curl -sS http://127.0.0.1:8000/health
```

Check the DTC route without printing the key:

```
cd /Users/divyesamdani/Documents/work/ev-fault-diagnosis-rag-full-integration
KEY=$(grep '^BACKEND_API_KEY=' backend/.env | cut -d= -f2-)
curl -i -H "Authorization: Bearer $KEY" \
  http://127.0.0.1:8000/api/v1/retrieval/dtc/P0A80
curl -i -H "Authorization: Bearer $KEY" \
  http://127.0.0.1:8000/api/v1/retrieval/dtc/P0100
```

Both should return `200 OK`.

Run tests:

```
cd /Users/divyesamdani/Documents/work/ev-fault-diagnosis-rag-full-integration
PYTHONPATH=agent-layer:.:scraper/vector_search \
  agent-layer/.venv/bin/python -m pytest -q \
  backend/tests agent-layer/tests scraper/tests/test_retrieval.py
```

Build the frontend:

```
cd frontend
npm run build
```

Configuration and security

Local `.env` files contain API keys and service secrets. They must never be committed or pasted into messages, screenshots, or GitHub issues.

Important local settings include:

```
HYBRID_RETRIEVAL_USE_MOCK=false
HYBRID_RETRIEVAL_API_BASE_URL=http://127.0.0.1:8000/api/v1/retrieval
HYBRID_RETRIEVAL_API_KEY=<local backend secret>
BACKEND_API_KEY=<same local backend secret>
DEFAULT_LLM_PROVIDER=gemini
DEFAULT_LLM_MODEL=gemini-3.5-flash-lite
GOOGLE_API_KEY=<local Gemini key>
```

If a key is exposed, rotate it immediately and replace it in the local `.env` file. Do not use `NEXT_PUBLIC_*` variables for secrets.

Current status

- Frontend, backend, agent, retrieval, Knowledge Base, history, Gemini synthesis, and DTC lookup are connected.
- Generic and EV-specific DTC lookups work through the live route.
- Backend and agent tests pass for the implemented flow.
- The frontend production build passes.
- The project is pushed to the feature branch but is not merged into the target branch.
- The corpus, embeddings, `.env` files, SQLite database, and local evaluation reports remain machine-local.

Important limitation

The system provides evidence-grounded technical assistance, not guaranteed repair instructions. High-voltage battery and inverter work should only be performed by qualified personnel using the vehicle manufacturer's safety procedure.